

ChatOps for networks: useful if it declares what it costs

ChatOps

Operations

17 September 2026 · 8 minute read

Network management platforms are bought for their capabilities and used for their main screen. ChatOps promises to fix that, and brings a problem of its own: an assistant that does not declare whether it consumes a language model turns your bill into a monthly surprise. This article is about doing it properly.

The powerful tool nobody opens

There is a familiar pattern in network management platforms: they are bought for their capabilities and used for their main screen. The rest of the modules exist, work, and nobody goes in. Not because they are badly built, but because reaching the answer requires knowing which tab it lives in, with which filters, and in what order.

The operator who needs to know whether a device is compliant, or which path a flow takes, or what changed since yesterday, has the information available and still opens a session to the device. It is faster for them, even if it is worse for the organisation: it leaves no trace, it is not repeatable, and it does not scale to anyone without their experience.

A capability that requires knowing where to look is only used by the people who already knew.

What ChatOps is, and what it is not

ChatOps means operating from a conversation: you ask in natural language and the platform answers with real data, runs the query, or proposes an action. The idea is old in the systems world and arrives late to networking.

It is worth saying what it is *not*, because the confusion is everywhere. It is not a language model answering from memory about networks: that produces plausible answers about a network that is not yours. Nor is it a terminal by another name.

A useful operations assistant does three distinct things, and they are worth separating because their risks are very different:

- **Query.** Read the inventory, compliance, topology. No risk: it only reads.
- **Diagnose.** Correlate several sources to explain something. Low risk, but it can be wrong, so it must show what it based the answer on.

- **Propose changes.** Generate configuration. Here the risk is real, which is why the answer cannot be the action: it has to be a proposal that someone approves.

The uncomfortable question: does this consume AI?

Anyone who has put a language model into production knows the two concerns: what it costs, and what leaves the organisation.

What almost nobody says is that **a large share of what an operator asks does not need a language model at all.** "How many devices are at the north site?" is a query. "Which devices fail the time server rule?" is a filter. "Which path does traffic take between these two addresses?" is a walk over routing tables. Solving those with a model is more expensive, slower, and less accurate than solving them deterministically.

The model earns its place where it genuinely adds value: interpreting an ambiguous question, writing an explanation, proposing configuration from an intent described in words.

An assistant that does not tell you whether it will consume the model before it runs turns your bill into a monthly surprise.

Declaring it *before* execution changes two things. The first is cost, which stops being a mystery. The second is trust: knowing an answer came from a deterministic query rather than a model tells the operator how much verification it needs.

A chat that can change the network needs guardrails

Conversing with the platform is convenient, and that convenience is precisely the risk: it lowers friction so much that requesting a change becomes as easy as asking the time. The protections have to be explicit.

- **Separate reading from writing.** Querying is immediate; changing goes through the same approval flow as any other change, with its reviewable diff.
- **Show the source.** An answer about the network must be able to point at where it came from: which device, which collection, from what date.
- **Respect the asker's permissions.** The assistant cannot become a back door to data that user would not see in the interface.
- **Leave a trace.** What was requested through conversation has to land in the same history as what was requested through the screen, or you lose the traceability that justified automating in the first place.

How Oktavia handles it

Argos is the conversation layer across the whole platform, not a separate module. It reaches inventory, compliance, topology, security, addressing, and remediation, so the question does not require knowing which screen the answer lives on.

Every action declares, **before it runs**, whether it will consume the language model or resolve deterministically. The operator decides with the cost in plain sight, and knows what kind of answer they are about to receive.

When Argos proposes configuration, it does not apply it: it produces the same proposal as the Digital Twin, with its reviewable diff and its approval, the flow described in [the article on closing the loop](#). Conversation changes how things are asked for, not the protections around them.

How many modules of your current platform do you actually use?

If the answer is "two or three", capability is not your problem. To quote, all we need is roughly how many devices you manage.

About Oktanet

Oktanet is a Mexican network automation company focused on controlled, traceable, and scalable operations for enterprise environments across Latin America.

[LinkedIn](#) · [YouTube](#)

© 2026 Oktanet · Oktavia · Public article, with no confidential customer information.